

多表查询

一、连接查询：一个查询同时涉及两个以上的关系

交叉连接查询
内连接查询
外连接查询

2种连接查询标准：SQL92标准、SQL99标准(✓)

1) 交叉连接 (又称广义笛卡尔积, 不带连接谓词, 无条件, 会产生无意义记录)

求学生表 student 和课程表 course 的广义笛卡尔积

SQL92: SELECT * FROM student, course;

SQL99: SELECT * FROM student CROSS JOIN course;

2) 内连接

内连接查询中用来连接两个表的条件称为连接条件或连接谓词

比较运算符主要有 $=$ $<>$ $>$ $<$ $>=$ $<=$ (仅使用 "=" 时为等值连接, 其余都是非等值连接)

连接条件中的列名是连接字段

连接条件中各连接字段必须可比, 但不必相同

① 等值连接

查询学生及其选修课程情况

SQL92: SELECT student.*, score.*
FROM student, score
WHERE student.sno = score.sno;

SQL99: SELECT student.*, score.*
FROM student JOIN score
ON student.sno = score.sno;

② 自然连接 (等值连接的两个表中有同名列)

查询学生及其选修课程情况

SQL99: SELECT * FROM student JOIN score USING(sno);
SELECT * FROM student NATURAL JOIN score;

③ 自身连接 (表与其自身连接, 为区别两表必取别名 (所有属性都是同名属性))

查询年龄大于王燕年龄的所有学生的姓名、系别、出生日期

SELECT b.name, b.dept, b.birthday

FROM student a JOIN student b

ON a.sno <> b.sno 别名

WHERE a.sname = '王燕' AND a.birthday > b.birthday;

3) 外连接

LEFT OUTER JOIN	除满足连接条件的行, 还包括左表所有行, 右表中数据无匹配值则为空
RIGHT OUTER JOIN	除满足连接条件的行, 还包括右表所有行, 左表中数据无匹配值则为空
FULL OUTER JOIN	包括左、右表中所有行

查询未选修任何课程的学生

```
SELECT *  
FROM student a LEFT JOIN ON score b  
ON a.sno = b.sno  
WHERE b.sno IS NULL;
```

二、嵌套查询

将一个查询块嵌套在另一个查询块中的查询

```
SELECT ...  
FROM ...  
WHERE ...
```

上层查询块称为外层查询/父查询

下层查询块称为内层查询/子查询

1) 不相关子查询 (子查询的查询条件不依赖于父查询)

① 标量子查询 (返回单值, 一行一列)

查询选修离散数学的学生的学号

```
SELECT sno FROM score
```

```
WHERE cno = (SELECT cno FROM course WHERE cname = '离散数学')
```

查询年龄大于王燕年龄的所有学生姓名、系别、出生日期

```
SELECT sname, dept, birthday FROM student
```

```
WHERE birthday < (SELECT birthday FROM student where sname = '王燕')
```

② 列子查询 (返回一组值, 一行多列)

查询选修离散数学的学生姓名

```
SELECT sname FROM student WHERE sno IN
```

```
(SELECT sno FROM score WHERE cno =
```

```
(SELECT cno FROM course WHERE cname = '离散数学'))
```

查询比所有计算机系学生年龄都大的学生

```
SELECT * FROM student
```

```
WHERE dept <> '计算机' AND birthday < ALL (SELECT birthday FROM student WHERE dept = '计算机')
```

使用 ANY/ALL 时必须同时使用比较运算符

③ 表子查询 (返回一张表, 多列多行)

查询计算机系总学分排名前3的学生信息

```
SELECT *
```

```
FROM (SELECT * FROM student WHERE dept = '计算机') AS qsm
```

```
ORDER BY totalcredit DESC
```

```
LIMIT 3;
```

子查询可在 FROM 后作数据源

用在DDL和DML中的子查询

① 用子查询结果创建表

将计算机系的所有学生信息保存到 stu 表中 (stu 未存在)

```
CREATE TABLE stu  
(SELECT * FROM student WHERE dept = '计算机');
```

② 将子查询结果保存到已有表

将计算机系的所有学生信息保存到 stu 表中 (stu 已存在)

```
INSERT INTO stu  
(SELECT * FROM student WHERE dept = '计算机');
```

切记 stu 表要与子查询结果结构相同!

③ 用子查询结果更新表

将 stu 表中所有学生的总学分更新为 student 表中的平均学分

```
UPDATE stu SET totalcredit =
```

```
(SELECT AVG(totalcredit) FROM student)
```

④ 用子查询结果删除表

从 stu 表中删除与 student 表中“王”姓同学学号相同的学生

```
DELETE FROM stu WHERE sno IN
```

```
(SELECT sno FROM student WHERE sname LIKE '王%');
```

2) 相关子查询

一般来说,子查询都在父查询前处理

而相关子查询不同,其子查询的查询条件往往依赖于父查询的某属性值

执行子查询的过程:

从外查询的关系中依次取一个记录,根据它的值在内查询中进行检查,

若WHERE子句中的条件表达式值为TRUE,则将此记录放入结果表;FALSE就舍去

直至外查询关系的记录全部被处理完

查询所有选修3102号课程的学生姓名

```
SELECT sname
```

```
FROM student
```

```
WHERE EXISTS ( SELECT * FROM score WHERE sno = student.sno AND cno = '102' );
```

将 score 与 student 两张表关联起来

查询选修了全部课程的学生姓名 ☆用 NOT EXISTS 实现除运算

↳ 转双重否定: 不存在任何一门课程,该学生没有选修

$$\Pi_{sname} (\text{student} \div (\Pi_{sno, cno} (\text{score}) \div \Pi_{cno} (\text{course})))$$

```
SELECT sname FROM student WHERE NOT EXISTS
```

```
(SELECT cno FROM course WHERE NOT EXISTS
```

```
(SELECT * FROM score WHERE sno = student.sno AND cno = course.cno));
```

三、传统的集合运算

MySQL 只支持(并)运算, 自动删除重复记录 (若要保留重复的, 加 ALL)

查询所有选修了 101 或 102 号课程的学生学号

```
SELECT sno FROM score WHERE cno = '101'
```

UNION

```
SELECT sno FROM score WHERE cno = '102';
```

其实也可以用之前学过的

```
SELECT DISTINCT sno  
FROM student  
WHERE cno IN ('101', '102');
```

已知客户表、代理商表结构如下:

customer (cid, cname, city, address)

agent (aid, aname, city, address)

查询客户及代理商所在城市:

```
SELECT cname, city FROM customer
```

UNION ALL

```
SELECT aname, city FROM agent;
```

练习.

查询比计算机专业女生还少的专业

```
SELECT dept, COUNT(*)  
FROM student  
WHERE sex = '女'  
GROUP BY dept  
HAVING COUNT(*) < (SELECT COUNT(*)  
FROM student  
WHERE dept = '计算机' AND sex = '女');
```

查询选修了‘史努比’同学选修的全部课程的学生们的姓名

双重否定: 不存在‘史努比’选的课程该同学没选

```
SELECT sname FROM student a  
a.sname <> '史努比' AND  
WHERE NOT EXISTS (SELECT cno FROM course  
(SELECT cno FROM student NATURAL JOIN score WHERE sname = '史努比') b  
WHERE NOT EXISTS (SELECT * FROM score WHERE sno = astudent.sno AND cno = bcourse.cno));
```

```
SELECT sname FROM student a  
a.sname <> '史' AND  
WHERE NOT EXISTS (SELECT cno FROM course  
(SELECT cno FROM student NATURAL JOIN score WHERE sname = '史') b  
WHERE NOT EXISTS (SELECT * FROM score WHERE sno = astudent.sno AND cno = bcourse.cno));
```